БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ЛАБОРАТОРНАЯ РАБОТА № 2

Чтение информации из графических файлов

Выполнил	Скворцов Илья
Форма реализации	Веб-приложение (HTML, CSS, JavaScript)

Минск, 2026

1. Цель работы

Закрепить знания о структуре растровых графических форматов и получить практические навыки извлечения метаданных непосредственно из байтов файла. Для достижения цели реализовать приложение, которое определяет параметры изображений, обрабатывает большие папки без загрузки файлов целиком в память и сообщает о повреждённых или некорректно размеченных файлах.

2. Постановка задачи и критерии

Разработать веб-приложение для выбора локальной папки и анализа графических файлов. Для каждого файла необходимо отобразить имя, формат, ширину и высоту в пикселях, разрешение (если оно записано в файле), глубину цвета, способ сжатия и результат проверки целостности.

Требование	Реализация в проекте
Форматы	JPEG/JPG, GIF, TIFF/TIF, BMP, PNG, PCX
Запрет сторонних парсеров метаданных	Метаданные разбираются собственным JavaScript-кодом; используются стандартные File/Blob API
Большие каталоги	Обход выбранной папки, очередь заданий и постраничный вывод по 100 строк
Отзывчивый интерфейс	Разбор выполняется в Web Workers, отдельно от основного потока интерфейса
Контроль состояния	Прогресс, число обработанных файлов и ошибок, фильтры и экспорт CSV
Повреждённые файлы	Проверка сигнатур и обязательных структурных маркеров форматов

3. Архитектура приложения

Решение разделено на три логических слоя, чтобы пользовательский интерфейс не зависел от деталей двоичного формата.

Слой	Файлы	Назначение
Presentation Layer	index.html, style.css	Разметка интерфейса, оформление, таблица, фильтры, прогресс и статистика.
Core / Orchestration	app.js	Приём выбранных файлов, очередь задач, запуск Worker-потоков, обновление таблицы, экспорт CSV.
Parsing Layer	worker.js	Определение формата по сигнатуре, чтение нужных диапазонов байтов, извлечение полей и валидация.

Основной сценарий: пользователь выбирает папку → приложение формирует очередь → Worker разбирает файл → результат возвращается в основной поток → интерфейс обновляет прогресс и таблицу.

4. Извлечение метаданных

Формат определяется по содержимому, а не только по расширению имени. Для чтения используются Blob.slice() и arrayBuffer(): парсер получает необходимые диапазоны, а не загружает всё содержимое каждого файла.

Формат	Поля и структуры, анализируемые парсером

JPEG/JPG	SOI, сегменты, SOF (размеры и компоненты), JFIF (разрешение), EOI FF D9.
GIF	GIF87a/GIF89a, Logical Screen Descriptor, глобальная таблица цветов, Trailer; сжатие LZW.
TIFF/TIF	Порядок байтов II/MM, magic 42, IFD и теги ширины/высоты, BitsPerSample, Compression, SamplesPerPixel, X/YResolution и ResolutionUnit.
BMP	BITMAPFILEHEADER и DIB-заголовок, ширина/высота, planes, bits-per-pixel, compression, палитра и смещение пиксельных данных.
PNG	Сигнатура, IHDR, bit depth, color type, pHYs, IDAT/IEND и CRC для проверяемых чанков.
PCX	128-байтовый заголовок, координаты, DPI, bits per plane, число плоскостей, bytes per line и тип кодирования.

5. Как интерпретируются характеристики

Размер изображения берётся из заголовка или обязательных тегов формата и выводится как ширина × высота в пикселях. Разрешение считывается из JFIF, pHYs, полей pixels-per-meter, TIFF-тегов либо полей DPI в PCX. Если файл не содержит физического разрешения, приложение показывает, что оно не задано, вместо подстановки вымышленного значения.

Глубина цвета отражает количество бит, используемых для представления пикселя или компонента, в зависимости от структуры конкретного формата. Поле сжатия показывает код, сохранённый в заголовке/теге, либо метод, предписанный форматом (например, Deflate для PNG, LZW для GIF). Значения разных форматов не всегда напрямую сопоставимы: их смысл зависит от спецификации.

6. Потоковая обработка и многопоточность

Для сокращения чтения с диска используются выборочные диапазоны файла. Например, у JPEG анализируются сегменты заголовка и отдельно последние два байта для проверки EOI; у TIFF читаются IFD и диапазоны, на которые ссылаются нужные теги. Задания распределяются между Web Workers, число которых ограничено сверху восемью. Это позволяет выполнять разбор вне основного потока интерфейса.

Результаты выводятся страницами по 100 записей, что ограничивает число DOM-элементов, одновременно создаваемых для таблицы. В интерфейсе отображаются процент выполнения, количество найденных файлов, обработанных записей, ошибок и прошедшее время.

7. Проверка целостности

Парсер проверяет сигнатуру и границы структур до чтения полей. В частности, проверяются PNG IHDR/IEND и CRC выбранных чанков, JPEG SOF и завершающий EOI, GIF Trailer, TIFF byte order/magic/смещения IFD, а также ключевые размеры и поля BMP/PCX. Ошибка разбора перехватывается и возвращается как статус «Файл поврежден» с пояснением; это предотвращает аварийное завершение всего сканирования.

Такие проверки повышают устойчивость, но не являются полной экспертизой всех возможных повреждений: для итоговой оценки нужно прогнать официальный тестовый набор и дополнительно проверить файлы с подменёнными расширениями.

8. Пользовательский интерфейс

Интерфейс включает выбор папки, кнопку очистки, прогресс-бар, сводные счётчики, поиск по имени, фильтры по формату и статусу, постраничную таблицу, просмотр доступного изображения и экспорт результатов в CSV. В таблице отдельно представлены формат, размеры, DPI, глубина цвета, сжатие, статус и поясняющие сведения.

9. Проверка и результаты

В комплекте проекта находятся тестовые архивы для проверки. В исходной версии отчёта зафиксированы следующие результаты запусков; они перенесены как ранее записанные измерения и не являются повторным замером в рамках подготовки этой редакции отчёта.

Набор	Ранее записанный результат	Что подтверждает
Архив преподавателя: 24 BMP	24/24 OK; 0 ошибок; 100%; около 0,8 с	Обработка набора BMP и вывод метаданных.
Дополнительный набор: 9 файлов	около 0,3 с	Проверка разных форматов, повреждённых и подменённых файлов.
Папка из 10 000 файлов	около 16,4 с	Работа на каталоге с большим количеством файлов.
600 JPEG, около 2,17 GB суммарно	600/600; 0 ошибок; 100%; около 1,1 с	Проверка выборочного чтения крупных файлов.

Перед сдачей следует повторить эти запуски на целевом компьютере и сохранить фактические скриншоты результатов. Время зависит от диска, браузера, числа Worker-потоков и состояния системы, поэтому приведённые значения не следует считать универсальной гарантией производительности.

10. Запуск приложения

1. Откройте папку lab2.
2. Для локальной проверки запустите локальный HTTP-сервер либо откройте страницу в браузере, поддерживающем используемые API и выбор каталога.
3. Нажмите «Выбрать папку» и укажите каталог с изображениями.
4. Дождитесь завершения обработки; изучите статус, размеры и дополнительные сведения.
5. Для сохранения результатов нажмите «Экспорт CSV».

Для публикации можно разместить файлы index.html, style.css, app.js и worker.js в GitHub Pages. Доступность выбора папки зависит от браузера; для проверки рекомендуется актуальная версия Chromium-браузера.

11. Соответствие дополнительным требованиям группы 10А

Требование	Статус по реализации
Ручной парсинг шести форматов без библиотек метаданных	Реализовано в worker.js; следует подтвердить официальным тестовым архивом.
Ленивое чтение заголовков/участков файла	Реализовано через Blob.slice(); отдельные форматы требуют ограниченного поиска маркеров.
Многопоточный режим и прогресс	Реализовано через Web Workers и прогресс-бар.
Обработка повреждённых и подменённых файлов	Есть проверки сигнатур и структурных маркеров; полноту следует подтвердить стресс-тестом.
Сложные TIFF-теги и крайние случаи	Реализован разбор IFD и перечисленных тегов; исчерпывающее покрытие всех вариантов TIFF не

	заявляется без отдельного тестирования.

12. Вывод

В ходе работы разработано веб-приложение для получения основной информации о растровых графических файлах. Приложение самостоятельно анализирует двоичные структуры JPEG, GIF, TIFF, BMP, PNG и PCX, не используя сторонние библиотеки для чтения метаданных. Выборочное чтение диапазонов и Web Workers уменьшают объём чтения и позволяют не выполнять разбор в основном потоке интерфейса. Реализованы отображение результатов в таблице, фильтрация, постраничная выдача, индикация прогресса, экспорт CSV и обработка ошибок разбора. Для подтверждения максимальной оценки необходим повторный прогон официального набора и документирование фактических результатов стресс-тестирования.

Приложение А. Структура проекта

- index.html — структура страницы;
- style.css — оформление интерфейса;
- app.js — управление очередью, Worker-потоками и отображением результатов;
- worker.js — ручные парсеры и проверки форматов;
- Tests/ — архивы и наборы для тестирования;
- README.md и TESTING_AND_FORMATS.md — инструкции и технические сведения.